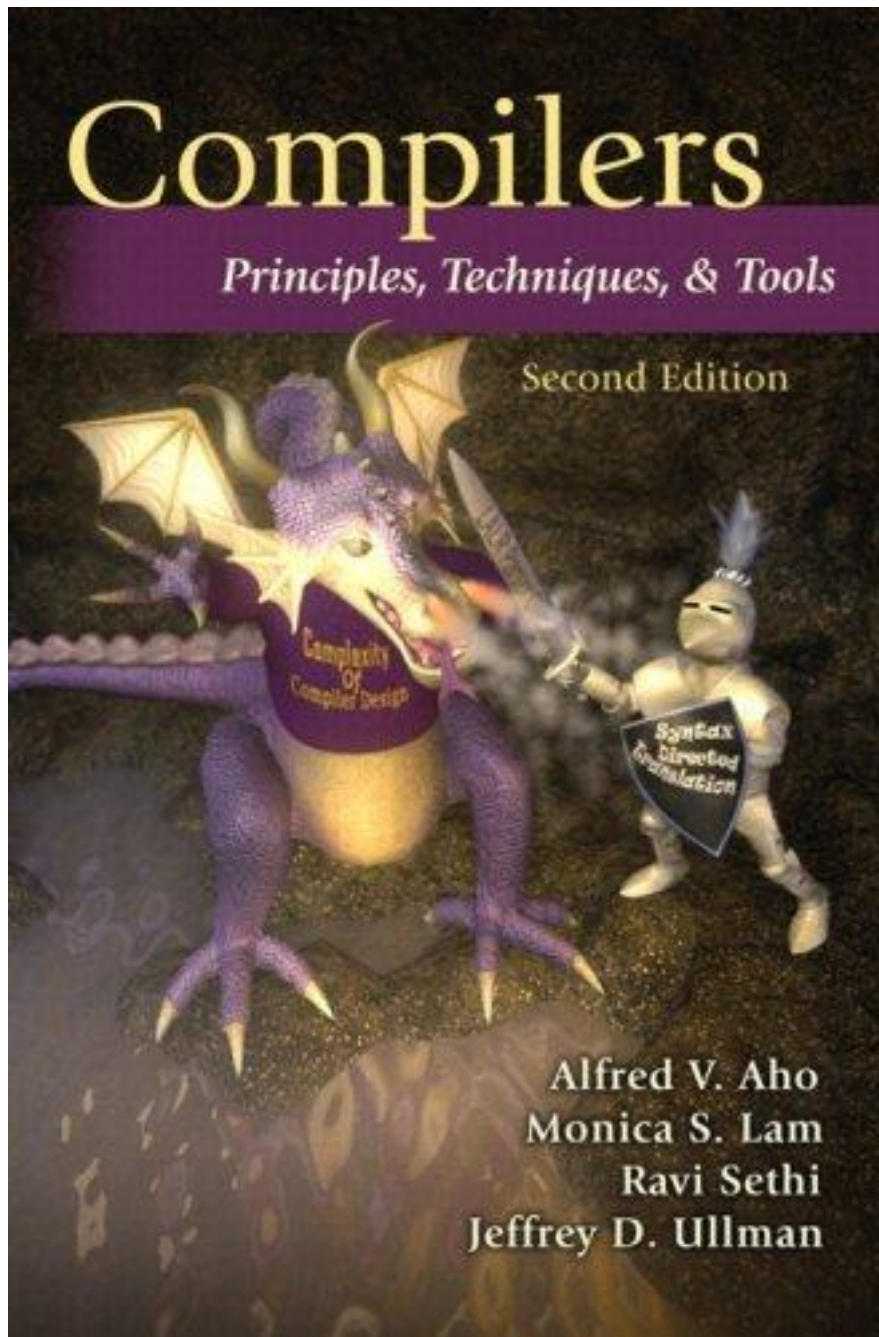




四川大學
SICHUAN UNIVERSITY

Principle of Compiler

luoyining@scu.edu.cn



Chapter 3

Lexical Analysis

From RE to FA

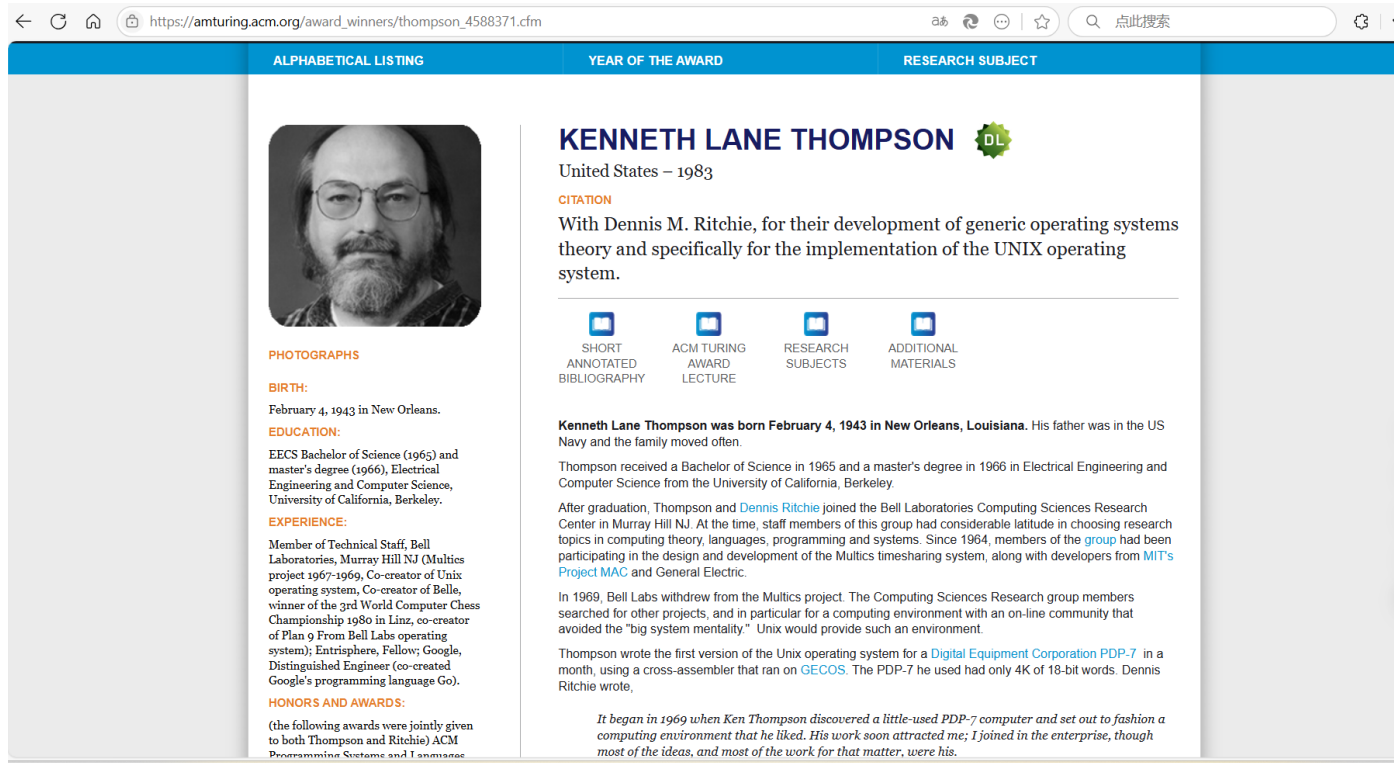
- Manual
 - From RE to NFA
 - From RE to DFA
- Auto
 - 3.7.4: $\text{RE} \rightarrow \text{NFA}$ (Thompson algorithm)
 - 3.7.1: $\text{NFA} \rightarrow \text{DFA}$ (Subset construction algorithm)
 - 3.9.6: Minimize DFA (Minimization algorithm)

3.7 From Regular Expressions to Automata



3.7.4 Construction of an NFA from a Regular Expression

- Thompson algorithm / Thompson's construction
(McNaughton-Yamada-Thompson algorithm)



The screenshot shows the ACM Turing Award page for Kenneth Lane Thompson. The page is titled "KENNETH LANE THOMPSON" and includes a portrait photo. The left sidebar contains sections for "PHOTOGRAPHS", "BIRTH:", "EDUCATION:", "EXPERIENCE:", and "HONORS AND AWARDS:". The main content area includes a "CITATION" section, a "RESEARCH SUBJECT" section, and a detailed biography. The biography mentions his work with Dennis M. Ritchie on the development of generic operating systems theory and the implementation of the UNIX operating system. It also notes his birth on February 4, 1943, in New Orleans, Louisiana, and his education at the University of California, Berkeley. The page is part of the "AMTURING" website, as indicated by the URL in the browser's address bar.

← ↻ 🏠 https://amturing.acm.org/award_winners/thompson_4588371.cfm 点此搜索

ALPHABETICAL LISTING YEAR OF THE AWARD RESEARCH SUBJECT

KENNETH LANE THOMPSON 

United States – 1983

CITATION

With Dennis M. Ritchie, for their development of generic operating systems theory and specifically for the implementation of the UNIX operating system.

PHOTOGRAPHS

BIRTH:
February 4, 1943 in New Orleans.

EDUCATION:
EECS Bachelor of Science (1965) and master's degree (1966), Electrical Engineering and Computer Science, University of California, Berkeley.

EXPERIENCE:
Member of Technical Staff, Bell Laboratories, Murray Hill NJ (Multics project 1967-1969, Co-creator of Unix operating system, Co-creator of Belle, winner of the 3rd World Computer Chess Championship 1980 in Linz, co-creator of Plan 9 From Bell Labs operating system); Entrisphere, Fellow; Google, Distinguished Engineer (co-created Google's programming language Go).

HONORS AND AWARDS:
(the following awards were jointly given to both Thompson and Ritchie) ACM Programming Systems and Languages

SHORT ANNOTATED BIBLIOGRAPHY

ACM TURING AWARD LECTURE

RESEARCH SUBJECTS

ADDITIONAL MATERIALS

Kenneth Lane Thompson was born February 4, 1943 in New Orleans, Louisiana. His father was in the US Navy and the family moved often.

Thompson received a Bachelor of Science in 1965 and a master's degree in 1966 in Electrical Engineering and Computer Science from the University of California, Berkeley.

After graduation, Thompson and [Dennis Ritchie](#) joined the Bell Laboratories Computing Sciences Research Center in Murray Hill NJ. At the time, staff members of this group had considerable latitude in choosing research topics in computing theory, languages, programming and systems. Since 1964, members of the [group](#) had been participating in the design and development of the Multics timesharing system, along with developers from [MIT's Project MAC](#) and General Electric.

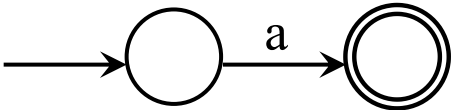
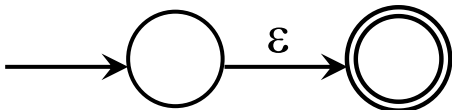
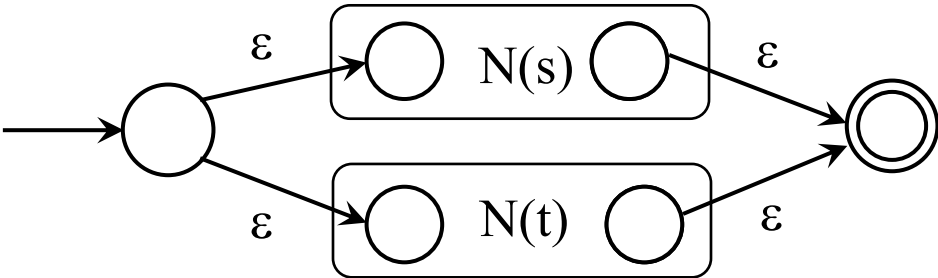
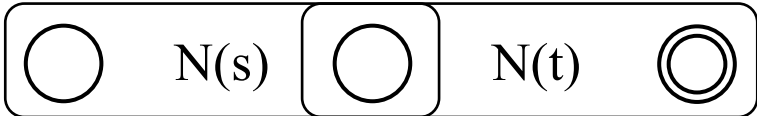
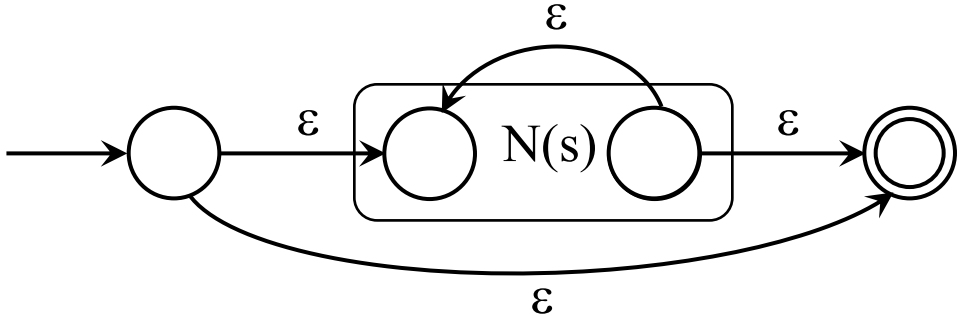
In 1969, Bell Labs withdrew from the Multics project. The Computing Sciences Research group members searched for other projects, and in particular for a computing environment with an on-line community that avoided the "big system mentality." Unix would provide such an environment.

Thompson wrote the first version of the Unix operating system for a [Digital Equipment Corporation PDP-7](#) in a month, using a cross-assembler that ran on [GECOS](#). The PDP-7 he used had only 4K of 18-bit words. Dennis Ritchie wrote,

It began in 1969 when Ken Thompson discovered a little-used PDP-7 computer and set out to fashion a computing environment that he liked. His work soon attracted me; I joined in the enterprise, though most of the ideas, and most of the work for that matter, were his.

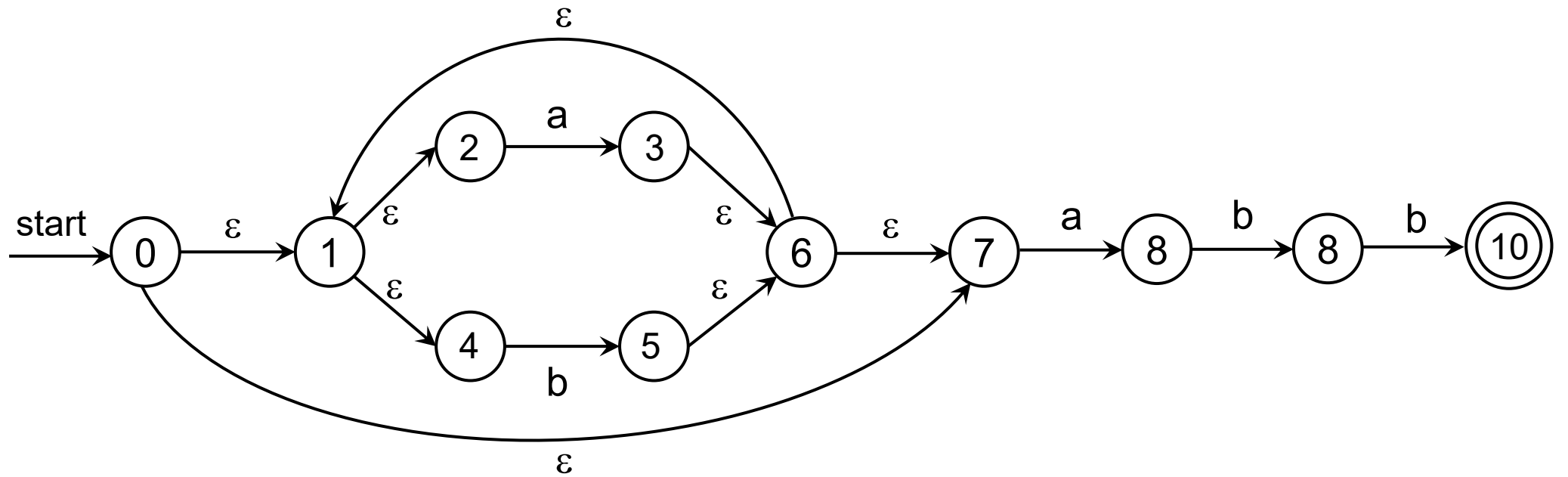
Algorithm 3.23

只有一个开始状态和一个接受状态。开始状态没有入边，接受状态没有出边。

Basis	a	 
	ε	
Induction	$r = s \mid t$	 $L(N) = L(N(s)) \cup L(N(t))$ $= L(s) \cup L(t) = L(s t) = L(r)$
	$r = st$	 $L(N) = L(N(s)) L(N(t))$ $= L(s)L(t) = L(st) = L(r)$
	$r = s^*$	 $L(N) = (L(N(s)))^*$ $= (L(s))^* = L(s^*) = L(r)$

Example 3.24

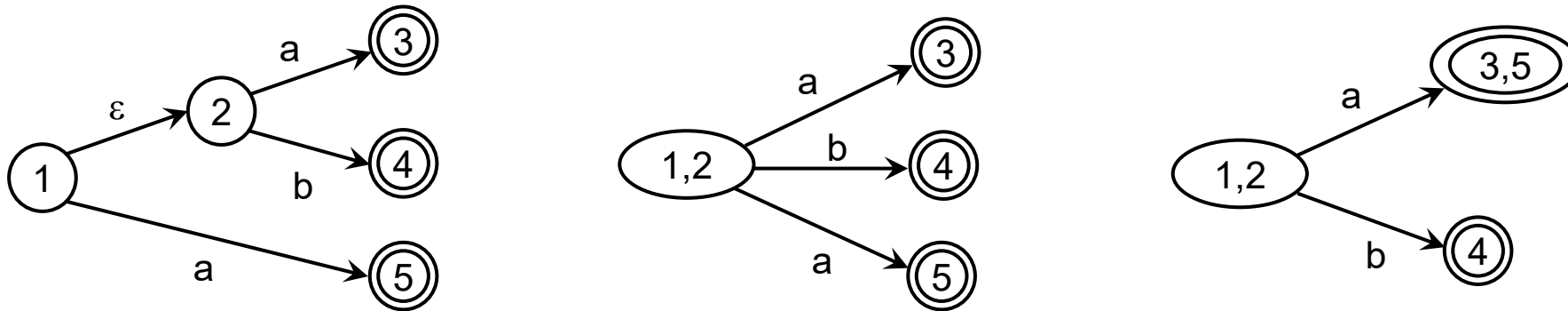
- $(a|b)^*abb$
 - Begin by parsing the RE into its constituent subexpressions.



- $((\epsilon|a)b^*)^*$

3.7.1 Conversion of an NFA to a DFA

- General idea
 - Difference between NFA : $S \times (\Sigma \cup \{\varepsilon\}) \rightarrow \rho(S)$ or 2^S and DFA: $S \times \Sigma \rightarrow S$
 - NFA $\rightarrow$ DFA, problems need to be resolved
 1. Eliminating ε -transitions
 2. Eliminating multiple transitions
- The general idea behind the [subset construction](#) is that
 - each state of the constructed DFA corresponds to a set of NFA states.



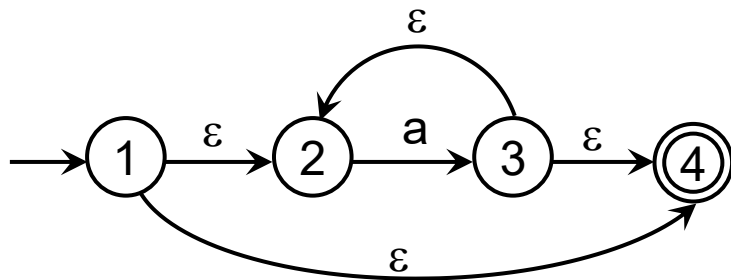
The DFA is equivalent to the NFA.

Definitions of Functions

- s : a single state T : state set a : a symbol in Σ

OPERATION	DESCRIPTION
$\epsilon\text{-closure}(s)$	Set of NFA states reachable from NFA state s on ϵ -transitions alone.
$\epsilon\text{-closure}(T)$	Set of NFA states reachable from some NFA state s in set T on ϵ -transitions alone; $= \bigcup_{s \in T} \epsilon\text{-closure}(s)$.
$\text{move}(T, a)$	Set of NFA states to which there is a transition on input symbol a from some state s in T .

Figure 3.31: Operations on NFA states



- set of states reachable by a series of **zero or more** ϵ -transitions from s .
- **union** of the $\epsilon\text{-closures}$ of each individual state in T .
- $\{ t \mid \text{for some } s \text{ in } T \text{ there is a transition from } s \text{ to } t \text{ on } a \}$

- $Dtran [T, a] = \epsilon\text{-closure}(\text{move}(T, a))$

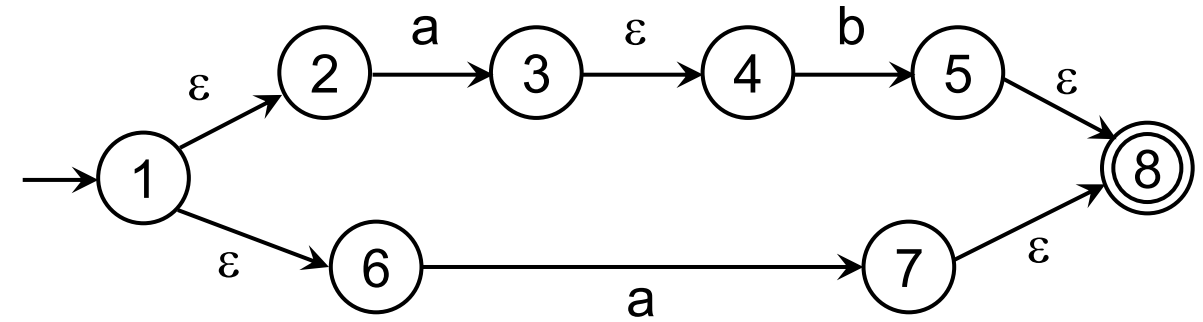
- or $I_a = \epsilon\text{-closure}(\text{move}(I, a))$

- $\epsilon\text{-closure}(1)$
- $\epsilon\text{-closure}(\{1, 2\})$
- $\epsilon\text{-closure}(\text{move}(\{2\}, a))$
- $\epsilon\text{-closure}(\text{move}(\{1, 2, 4\}, a))$

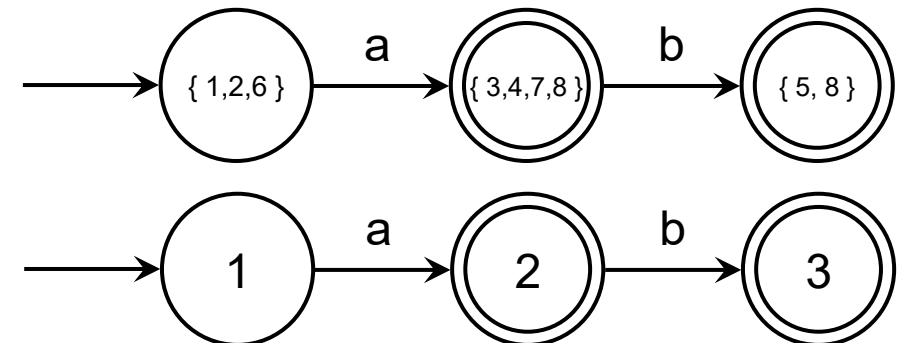
Algorithm 3.20 – Subset Construction

- Construction $\langle S, \Sigma, f, s_0, F \rangle$
 - set of DFA's states
 - transition function
 - $Dtran[T, a] = \varepsilon\text{-closure}(\text{move}(T, a))$
 - $I_a = \varepsilon\text{-closure}(\text{move}(I, a))$
 - s_0 : $\varepsilon\text{-closure}$ of start state of NFA
 - F : include at least one accepting state of NFA
- When/Can terminate?

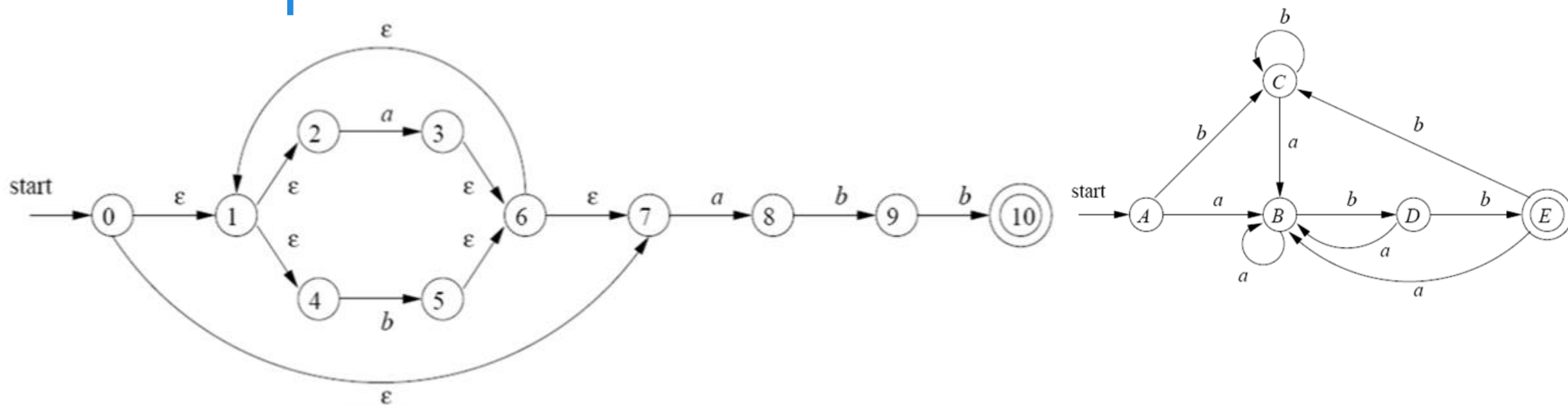
S	a₁	...	a_k
s_0	$Dtran[T, a_1]$		$Dtran[T, a_k]$
.....			



S	a	b
{1,2,6}	{3,4,7,8}	/
{3,4,7,8}	/	{5,8}
{5,8}	/	/



Example 3.21



Set of NFA States	DFA State	a	b

- 判断一个状态集合是否是DFA中的新状态集合，只需比较 $move$ 函数的结果即可，不需要比较 $Dtran / I$ 函数值。

A: $move(A,a) = \{3,8\}$, $I_a = \{1,2,3,4,6,7,8\}$; $move(A,b) = \{5\}$, $I_b = \{1,2,4,5,6,7\}$

B: $move(B,a) = \{3,8\}$, $I_a = \{1,2,3,4,6,7,8\}$; $move(B,b) = \{5,9\}$, $I_b = \{1,2,4,5,6,7,9\}$

C: $move(C,a) = \{3,8\}$, $I_a = \{1,2,3,4,6,7,8\}$; $move(C,b) = \{5\}$, $I_b = \{1,2,4,5,6,7\}$

D: $move(D,a) = \{3,8\}$, $I_a = \{1,2,3,4,6,7,8\}$; $move(D,b) = \{5,10\}$, $I_b = \{1,2,4,5,6,7,10\}$

E: $move(E,a) = \{3,8\}$, $I_a = \{1,2,3,4,6,7,8\}$; $move(E,b) = \{5\}$, $I_b = \{1,2,4,5,6,7\}$

Exercise 3.7.1

- Convert to DFA's the NFA's of:

a) Fig. 3.26.

b) Fig. 3.29.

c) Fig. 3.30.

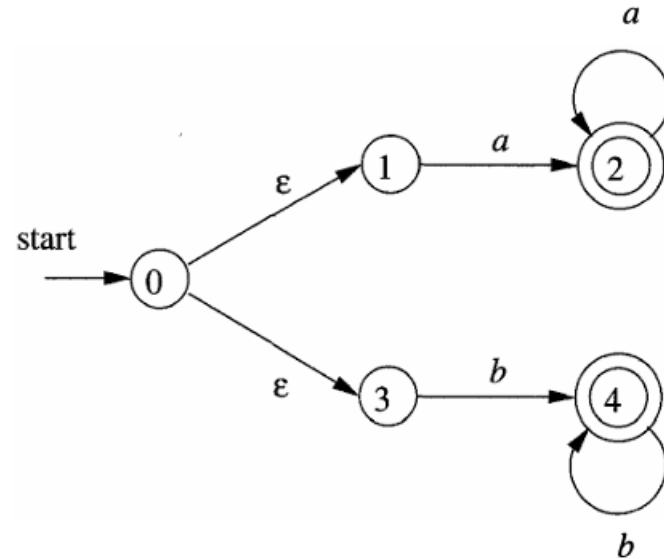


Figure 3.26: NFA accepting $aa^*|bb^*$

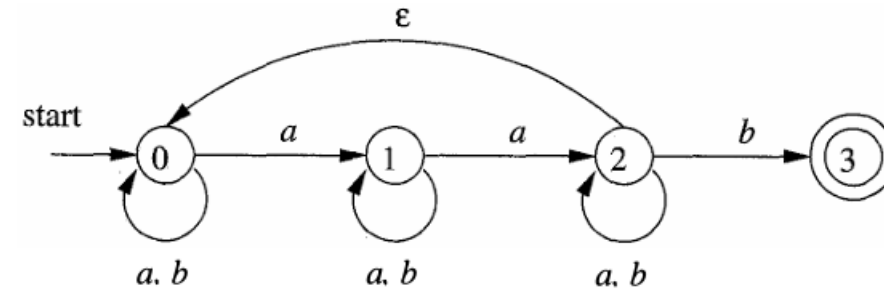


Figure 3.29: NFA for Exercise 3.6.3

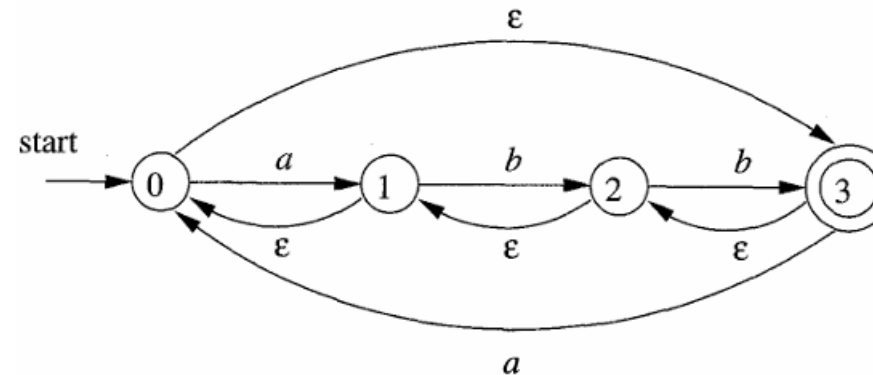
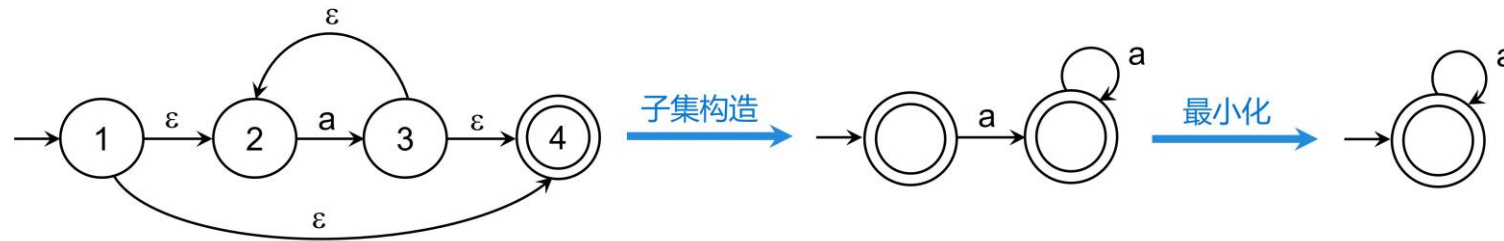


Figure 3.30: NFA for Exercise 3.6.4

3.9.6 Minimizing the Number of States of a DFA

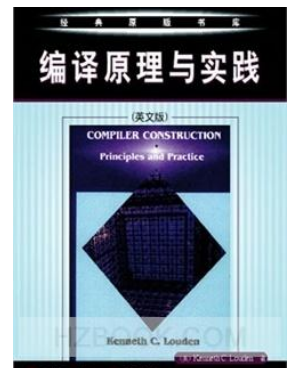
- Why Minimize?

- The resulting DFA constructed by subset construction may be more complex than necessary.



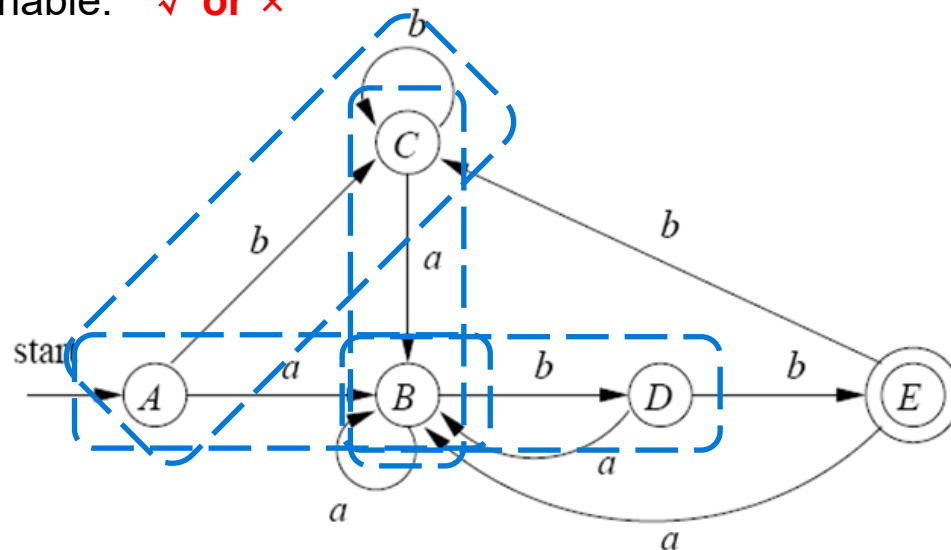
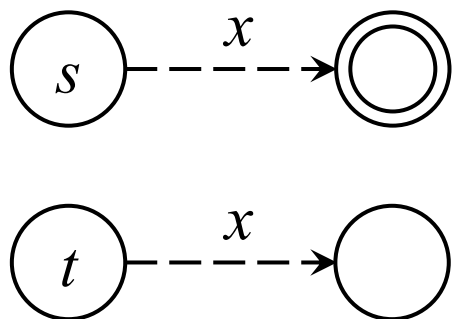
- 自动机理论中有一个很重要的结论：

对于任何DFA，都有一个包含最小状态的等价的DFA，而且这个最小状态的DFA是唯一的。



Distinguishable vs. Equivalent

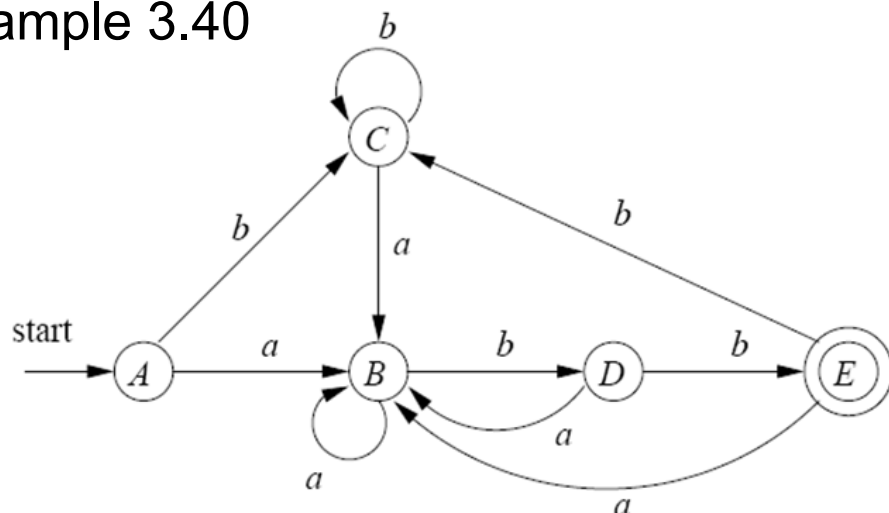
- 如果分别从状态 s 和 t 出发，沿着标号为 x 的路径到达的两个状态中只有一个是接受状态，则串 x 区分状态 s 和 t 。
- 如果存在某个能够区分状态 s 和 t 的串，则 s 和 t 是可区分的。
- Accepting state and nonaccepting state are distinguishable. ✓ or ✕



- 在有穷自动机中，两个状态 s 和 t 等价的条件是：
 1. 状态 s 和 t 必须同时为可接受状态或不可接受状态。 (一致性条件)
 2. 对于所有输入符号，状态 s 和 t 必须转换到等价的状态里。 (蔓延性条件)

Algorithm 3.39 – Minimization

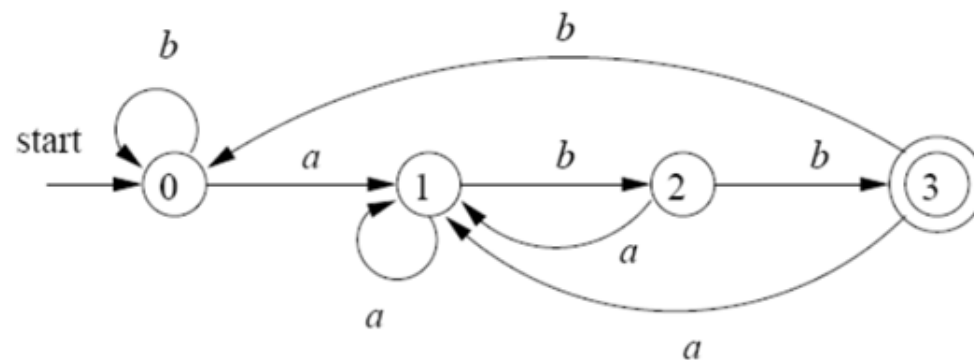
- Example 3.40



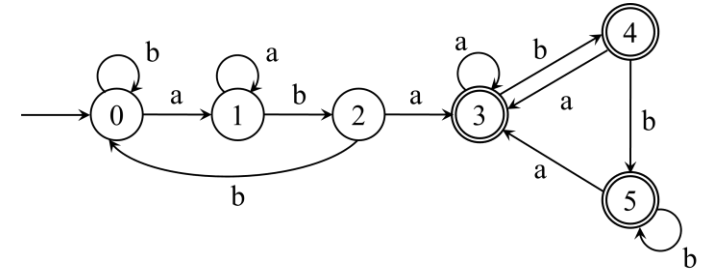
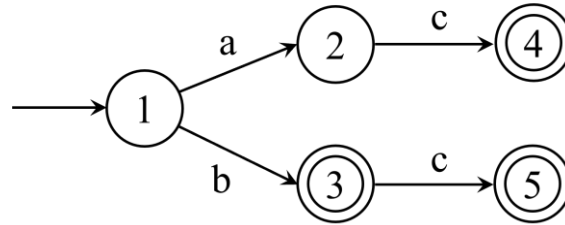
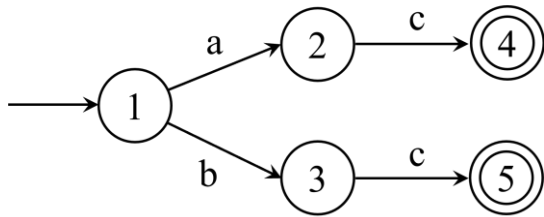
1. 把所有状态划分为两个不相交的子集：
接受状态集和非接受状态集；
2. 若某个子集内所有状态都等价，则不再划分；
若不等价则划分；
3. 重复步骤2，直到每个子集都不能再划分，转步骤4；
4. 确定开始状态和接受状态。

STATE	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

$\{A, B, C, D\} \quad \{E\}$
 $\downarrow a, b$
 $\{A, B, C\} \quad \{D\} \quad \{E\}$
 $\downarrow a, b$
 $\{A, C\} \quad \{B\} \quad \{D\} \quad \{E\}$
 $\downarrow a, b$
 $\{A, C\} \quad \{B\} \quad \{D\} \quad \{E\}$

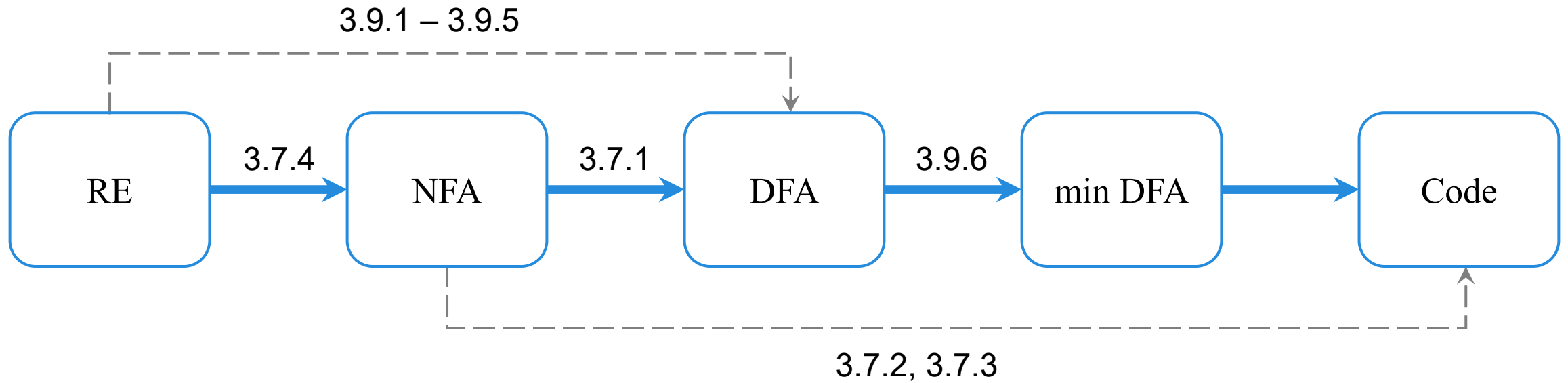


Exercises



- Exercise 3.9.3** We can prove that two regular expressions are equivalent by showing that their minimum-state DFA's are the same up to renaming of states. Show in this way that the following regular expressions: $(a|b)^*$, $(a^*|b^*)^*$, and $((\epsilon|a)b^*)^*$ are all equivalent. Note: You may have constructed the DFA's for these expressions in response to Exercise 3.7.3.
- ! Exercise 3.3.2** Describe the languages denoted by the following regular expressions:
 - $a(a|b)^*a$.
 - $((\epsilon|a)b^*)^*$.
 - $(a|b)^*a(a|b)(a|b)$.
 - $a^*ba^*ba^*ba^*$.
 - !! $(aa|bb)^*((ab|ba)(aa|bb)^*(ab|ba)(aa|bb)^*)^*$.
- All strings of decimal integers that are divisible by 5.

From RE to FA/Code



完整的有限状态机程序 如下程序实现图 6-5 所示的有符号整数 FSM:

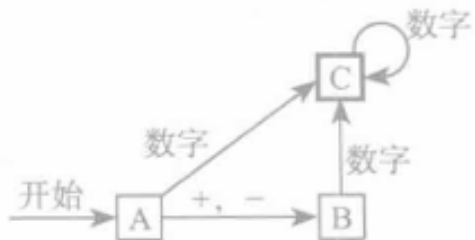


图 6-5 有符号十进制整数
FSM

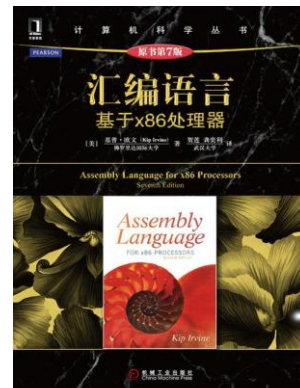
```
; 有限状态机                                     (Finite.asm)
INCLUDE Irvine32.inc
ENTER_KEY = 13
.data
InvalidInputMsg BYTE "Invalid input",13,10,0
.code
main PROC
    call Cclrscr

StateA:
    call Getnext                                ; 读取下一个字符, 并送入 AL
    cmp al, '+'                                ; 前置 + ?
    je StateB                                  ; 到状态 B
    cmp al, '-'                                ; 前置 - ?
    je StateB                                  ; 到状态 B
    call IsDigit                                ; 如果 AL 包含数字, 则 ZF=1
    jz StateC                                  ; 到状态 C
    call DisplayErrorMsg                        ; 发现非法输入
    jmp Quit

StateB:
    call Getnext                                ; 读取下一个字符, 并送入 AL
    call IsDigit                                ; 如果 AL 包含数字, 则 ZF=1
    jz StateC                                  ; 到状态 C
    call DisplayErrorMsg                        ; 发现非法输入
    jmp Quit

StateC:
    call Getnext                                ; 读取下一个字符, 并送入 AL
    call IsDigit                                ; 如果 AL 包含数字, 则 ZF=1
    jz StateC                                  ; 到状态 C
    cmp al, ENTER_KEY                          ; 按下 Enter 键?
    je Quit                                    ; 是: Quit
    call DisplayErrorMsg                        ; 否: 发现非法输入
    jmp Quit

Quit:
    call Crlf
    exit
main ENDP
```



End of Chapter Three